State Model

University Student-Support Case Agent

Group M Evening | BSE4104 | Week 6

Item	Detail
Project	University Student-Support Case Agent
Scope	Explicit workflow / session state for the bounded agent
Implementation	`src/agent/state.py`, `src/app.py`, `src/tools/orchestrate.py`
Related store	`data/cases/drafts.json` (draft records; not full chat memory)

1. Purpose

This state model makes the agent's working memory **explicit, inspectable and bounded**.

It answers:

- what the system remembers during a chat;
- what resets every student turn;
- what survives for the whole browser session;
- what is written outside the chat (draft file);
- what the model is **not** allowed to set or invent.

Week 5 introduced session state for the bounded loop. Week 6 keeps that model.
Case History Recall is the persistent-memory use, specified in the Memory Design Note.
This document is the state baseline those memory decisions build on.

2. State layers

The system uses three layers. Only Layer A and B are "agent state" in code today. Layer C is the durable draft store owned by deterministic software. Figure 1 (next page) shows how the layers, the orchestration loop, the foundation model and the human hand-off connect.

Layer	Lifetime	Where it lives	Cleared when
A. Turn state	One student message	Fields on `SessionState` reset each turn	Start of next turn / end of turn processing
B. Session state	One Streamlit browser session	`st.session_state.agent_state` (`SessionState`)	Browser session ends / app reload
C. Draft store	Until file deleted / replaced	`data/cases/drafts.json`	Manual retention policy (see Memory Note)

Week 6 state model

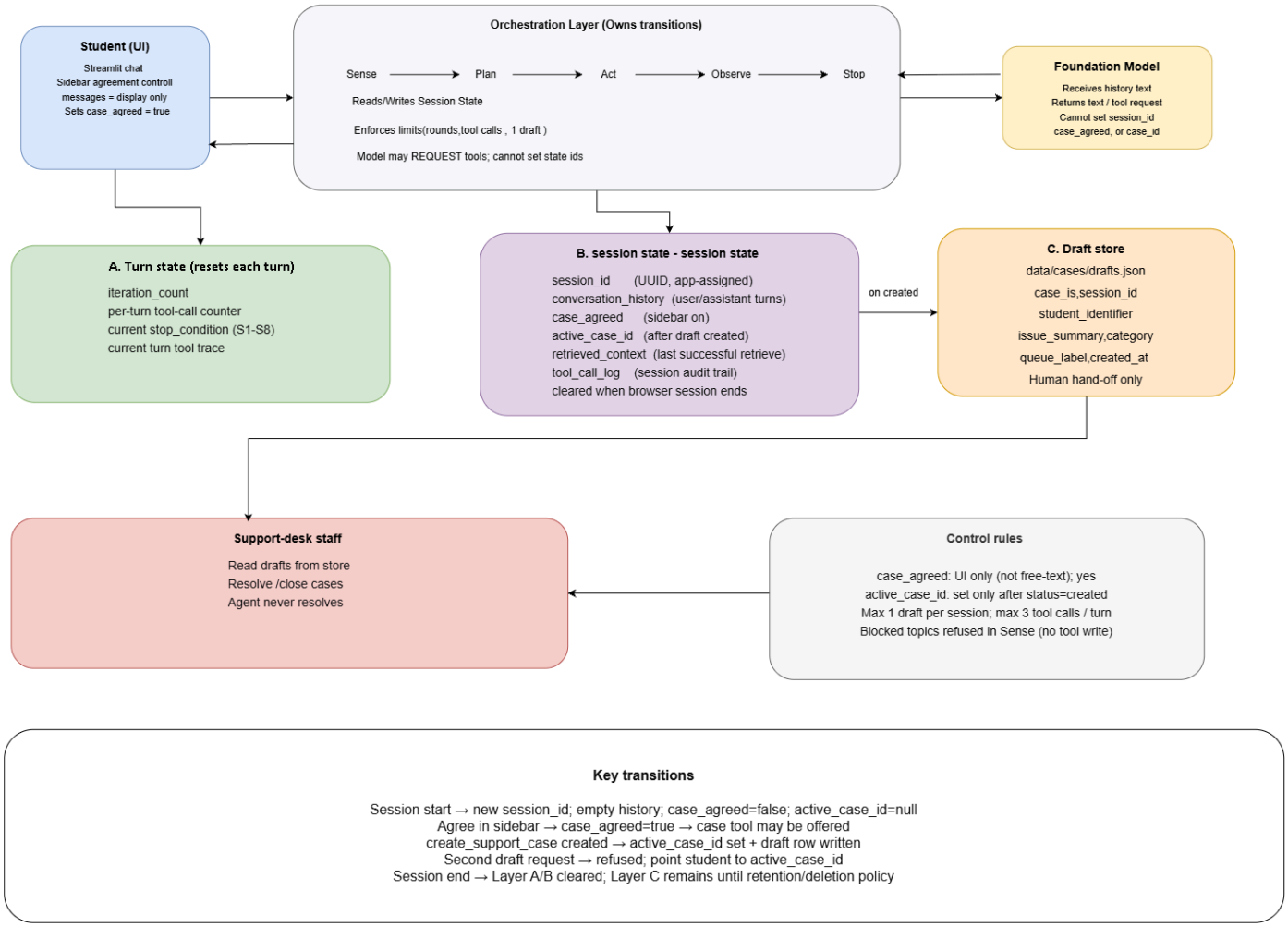


Figure 1. State model: Turn state (A) → Session state (B) → Draft store (C) → Support-desk staff.

3. Session state catalogue (`SessionState`)

All fields below are owned by **application code**. The foundation model may read conversation context the app chooses to send; it must not invent `session_id`, `case_agreed`, or `case_id`.

Field	Type	Layer	Written by	Read by	Purpose
`session_id`	UUID string	B	App (once at session start)	Case tool, traces	Ties drafts to this chat; never model-set
`conversation_history`	list of `{role, content}`	B	Orchestration after each user/assistant turn	Sense / Plan (model context)	Follow-ups without repeating the whole question
`case_agreed`	bool	B	UI sidebar button only	Tool allow-list / `_run_call`	Gate for `create_support_case`
`active_case_id`	string or null	B	Orchestration when draft `status=created`	Allow-list, prompts, follow-ups	Max one draft/session; answer "what's my case id?"
`retrieved_context`	list of chunk dicts	B	Orchestration after successful retrieve	Traces	Kept for traces. Next turn searches again.
`iteration_count`	int	A/B	Orchestration (+1 per model call); reset each turn	Stop condition S7	Enforce max 3 model rounds / turn
`tool_call_log`	list of `{timestamp, tool, arguments, result}`	B	Orchestration after every tool attempt	Traces, audit	Session audit trail

Turn-local items (not stored on SessionState as durable fields)

Item	Lifetime	Purpose
Per-turn tool-call counter	Current turn only	Enforce max 3 tool calls / turn
Current stop condition (S1–S8)	Returned with the reply	Explain why the turn ended
Current turn tool trace	Returned with the reply; also appended into `tool_call_log`	UI captions + Week 5/7 evidence

UI display list (separate from agent state)

Item	Notes
`st.session_state.messages`	Chat display list in Streamlit. Not the authoritative agent history. Authoritative history for the model is `conversation_history` inside `SessionState`.

4. Draft-store record shape (Layer C)

Created only by `create_support_case` → `case_store.create_draft`. One successful draft per session is linked via `active_case_id`.

Field	Set by	Notes
`case_id`	App (`CASE-YYYY-NNNNNN`)	Never trusted from the model
`session_id`	App	From `SessionState.session_id`
`student_identifier`	Validated input	2 digits, U or X, 4 or 5 digits, optional /PS or /EVE

Field	Set by	Notes
issue_summary	Validated input	Short onboarding issue text
category	Validated enum	Maps to queue_label
queue_label	Deterministic map	e.g. ict_helpdesk
created_at	App (UTC ISO-8601)	Audit timestamp

This file is the **human hand-off** artefact. The agent does not resolve or close cases.

5. Lifecycle and transitions

Session start

1. Create SessionState with new session_id.
2. All flags false / empty; active_case_id = null.

Each student turn

1. **Reset turn counters** (iteration_count = 0).
2. Append user message to conversation_history.
3. **Sense:** read flags + message; may stop with S4/S5/S6 before tools.
4. **Plan / Act / Observe:** model rounds; update iteration_count, tool_call_log, maybe retrieved_context.
5. On draft created: set active_case_id; append Layer C record.
6. Append assistant reply to conversation_history; return stop label.

Important transitions

Event	State change
Sidebar "I agree to save a support draft"	case_agreed = true
Successful create_support_case	active_case_id = case_id; draft row in drafts.json
Second draft requested same session	Tool refused; student pointed to active_case_id
retrieve_documents returns ok	retrieved_context replaced with new chunks
retrieve_documents returns no_match	No write to active_case_id; turn may stop at S3 and offer draft
Browser session ends	Layer A/B cleared; Layer C drafts remain on disk until retention policy applies

6. Who may change what (control boundary)

Item	Model may request?	Model may set directly?	App enforces
Tool call	Yes	No	Allow-list + limits
case_agreed	No	No	Sidebar only
session_id / case_id	No	No	App-generated
active_case_id	No	No	Set only after status=created
Conversation text	Produces assistant text	No arbitrary history rewrite	App appends turns

Item	Model may request?	Model may set directly?	App enforces
Admissions / grading / discipline / fee decision	Must refuse	—	Sense + tool guards

7. What this state model deliberately excludes

- Cross-browser-session chat memory (Week 6 memory feature is a **justified subset**, not full history replay).
- Real university SIS / email / live ticket APIs.
- Autonomous case resolve/close.
- Trusting free-text "yes" as consent.
- Letting the model invent identifiers.

8. Relation to Week 6 memory work

Already in state (baseline)	Case History Recall
Session fields above	New chat reads the old draft from drafts.jso only after the student gives their registration number again. Reload clears active_case_id.
`drafts.json` as hand-off file	Document access, retention, deletion in the Memory Design and Data Handling Note
Demo: follow-up in same session using `active_case_id`	Demo: memory improves a legitimate task without controlling critical decisions

The Memory Design Note names this use: Case History Recall. The old draft stays in drafts.json.
A reload clears the session. The case id does not come back on its own.

9. Acceptance checks for this deliverable

This state model is complete for Week 6 when it shows:

1. Explicit turn vs session vs draft-store layers (Section 2 and Figure 1).
2. Every `SessionState` field with owner, reader and lifetime (Section 3).
3. Transitions for agreement, draft creation, and session end (Section 5).
4. Clear boundary: model requests tools; app owns state (Section 6).
5. Pointer to the separate Memory Design and Data Handling Note for persistence policy (Section 8).